



r/rust • 6 yr. ago

Code-Sandwich



Why Rust uses glibc and not musl by default for Linux target?

Rust by default is compiled dynamic-linked with glibc, but it can be also static-linked with musl. The other version is more portable and probably could use LTO. Does usage of glibc produce faster/smaller/more feature complete/more stable binaries?

↑ 33 ↓

○ 37



↗ Share



Grammarly • Official • Promoted



With just a couple of clicks, Grammarly takes whatever's on the tip of your tongue and turns it into sentences—helping you create thoughtful email replies, content, and upvote-worthy Reddit posts instantly. Try it for free now.

Download

grammarly.com



+ Add a comment

Sort by: Best ▾



[deleted] • 6y ago

You can't swap out glibc without rebuilding every other third party library you want to link against that was built with glibc, that means e.g. pretty much everything in /usr/lib

↑ 46 ↓ ○ Reply ...

⊕ 4 more replies



dremon_nl • 6y ago

musl doesn't have certain features (for example no dlopen support) and so can't be a full replacement to dynamic linking.

↑ 9 ↓ ○ Reply ...



rust48383638 • 6y ago

Out of curiosity, I recompiled my rust binary (a numerical simulation) and it ran 9x slower with musl vs the default glibc linking. I haven't profiled the difference, but I'm guessing it's in libm.

↑ 5 ↓ ○ Reply ...



⊖ ↑ 3 ↓ 💬 Reply ...



Saefroch · 6y ago

Your binary can have no load/link-time dependencies. I don't know how often it is for people to run into these issues in professional software development but it's a constant pain for me.

I have a development machine that runs a rolling-release distro, and machines I want to run those programs on that... aren't. For reasons (some of them decent ones). If I naively build then copy over a `--target=x86_64-unknown-linux-gnu` binary, one will discover that the machine I want to run on has an incompatible version of openssl, and that the GLIBC load/run-time check is gonna fire and tell me that I have the wrong version.

Static linking and docker are the only fixes for this that I'm aware of.

I often seen this response to people (like me) who really want static linking

But you dynamic linking lets you get/do security updates without recompiling!

While this may be *technically* true I'm not convinced it's helpful. For this system to work both of these must be true:

- The security update is ABI-compatible; not only do the developers release an update that fixes security problems without breaking their ABI *as you use it*, the responsible fix isn't "Remove this symbol from our ABI, it was never a good idea."
- You actually get a new dynamic library on the system in a meaningful time frame. I'm no security expert but the latest Ubuntu-provided version of `openssl` (1.1.0g) isn't available on the openssl website, and instead openssl offers a newer version (1.1.0j and 1.1.1a) and mentions "bug and security fixes". Is your app going to be exploitable for a long time until Ubuntu ships you a fix? Could you do better?

⊖ ↑ 14 ↓ 💬 Reply ...



belovedeagle · 6y ago



Building on your second point, I think given today's linux software distribution practices, dynamic linking is likely to *prevent* getting security updates. When you have a bunch of packages on a (semi-)production system, are you going to want to:

1. take a point release on a dynamic library which could impact who knows how many applications in unknown ways (or, perhaps, just never get released until your distro maintainers verify this doesn't happen); or,
2. take a point release on specific static linked applications, where you know exactly what's going to update, and that the combination of libraries linked by that application was actually tested, and the update can be released independently of readiness of other packages.

What happens in practice is neither of these things: in practice, only the most critical software gets updated between major releases, if ever. Most users just install Ubuntu 18.10 or whatever and don't update until the next release (or they install an LTS, but those basically get updated as stable versions come out so the same limitations apply).

↑ 15 ↓ Reply ...

4 more replies



Rothon · 6y ago

Ubuntu manages their own copy of openssl and backport security fixes to it, FYI.

↑ 4 ↓ Reply ...

1 more reply



tomwhoiscontrary · 6y ago



that the GLIBC load/run-time check is gonna fire and tell me that I have the wrong version.

Okay, there are two really different things going on here.

One is that you have dynamically linked OpenSSL (or some other third-party library). This is definitely a pain when trying to build portable software! Static linking is useful here. Fortunately, this has nothing at all to do with whether stdlib is statically or dynamically linked; you can control it by setting the [OPENSSL_STATIC environment variable](#) when building.

The second is that you have dynamically linked libc. This, fundamentally, is absolutely fine. libc, and some related libraries like libm, librt, libpthread, etc, are part of the operating system, not part of your application. It is entirely appropriate to link to them dynamically. Indeed, it's only on Linux that it is even conceivable to link to them statically - on other operating systems, the libc is the only supported interface to the kernel, and you need to use the one supplied by the environment you're running in. And it's absolutely fine to dynamically link to libc etc while statically linking to third-party libraries!

Your only problem is if you link to symbols in libc which are available on your build machine, but not your deployment machine. Because glibc symbols are versioned, this can happen if you link to a version of a common symbol which is available on the build machine but not the deployment machine - memcpy@GLIBC_2.14 and memcpy@GLIBC_2.2.5 are not the same! Obviously, if your program uses a symbol which just isn't there, you're stuffed, but if it uses one where there's a version difference, you should be able to compile it to call the older version. This is where my knowledge runs out - i know that in C, you could use a `symver` directive to the assembler to pick a specific version of a symbol that you knew was widely available, but i have no idea if you can do this in Rust.

Looking at the pre-compiled Rust stdlib, that seems to link to very conservative versions - the one that comes with Rustup's 1.31.0 links to no symbol newer than GLIBC_2.4 (and uses the GLIBC_2.2.5 version of the [infamous](#) memcpy). So you should be fine if you only touch libc via stdlib.

But what if you use libc directly, or via some third-party library (eg [libc!](#))? The only thing i can think of is to find some old version of libc, and use that in your build process, so you get the right symbols :(.

↑ 2 ↓ Reply ...

+ 1 more reply



Lev1a · 6y ago

IIRC, static linkings advantage is that you can run the binary on any system that has the appropriate system libraries/architecture. Meaning you're not dependent on some dynamically linked library being on the machine of the end user.

Disadvantage: larger binaries, since everything is baked into the binary. This can be bad when you have many individual binaries on one system, all of which use some statically linked library, which in theory could also be installed once and then used by the binaries, resulting in filesize reductions when using it in a dynamically linked fashion.

Other disadvantages:

- Cannot update a linked library except by recompiling the executable, which makes it harder to fix bugs and security problems with the library. This is also transitively true of the libraries these libraries depend on.
- Some libraries just aren't shipped by some distros as static libraries at all, making it impossible to use them in a statically-linked binary. Heck, some library codebases have no reasonable way to build a static library.

The UNIX/Linux world has pretty much left static binaries behind, with a few small odd exceptions. It's really pretty much only an embedded thing now. (I am not a Windows person, but I have heard it's all still a mess there, some kind of DLL Hell if I understand correctly.)

↑ 14 ↓ Reply ...

+ 12 more replies



rabidferret · 6y ago

you can run the binary on any system that has the appropriate system libraries
you're not dependent on some dynamically linked library

Uhhhhhhh

↑ 1 ↓ Reply ...

+ 1 more reply



Mount_Everest · 6y ago

Calling out to a dynamically linked library is slow, for example: <http://ewontfix.com/18/>

⊖ ↑ 4 ↓ Reply ...



[deleted] · 6y ago

but certainly for a library like glibc, or whatever c std lib is in use, there are steps taken to avoid this problem? most binaries will need the c std lib

↑ 2 ↓ Reply ...

+ 2 more replies

New to Reddit?

Create your account and connect with a world of communities.



Continue with Google



Continue with Email



By continuing, you agree to our [User Agreement](#) and acknowledge that you understand the [Privacy Policy](#).



r/rust • 2 yr. ago

Updating Rust's Linux musl targets | Rust Blog

237 upvotes · 13 comments



r/rust • 4 yr. ago

Rust, musl and glibc in 2021

59 upvotes · 23 comments



r/rust • 4 yr. ago

is there a musl implementation in Rust ?

15 upvotes · 9 comments



r/Gentoo • 1 yr. ago

Musl vs Glibc

15 upvotes · 22 comments



r/rust • 4 mo. ago

Why Rust and not C?

71 comments



r/rust • 4 yr. ago

Why not always statically link with musl?

146 upvotes · 94 comments



r/rust • 2 yr. ago

rust on older (glibc 2.12) systems

8 upvotes · 23 comments



r/rust • 4 yr. ago

Building Rust binaries in CI that work with older GLIBC

149 upvotes · 48 comments



13 upvotes · 38 comments



r/rust · 3 yr. ago

Can rust be entirely written in rust and drop C usage in its code base ?

48 upvotes · 106 comments



r/rust · 2 mo. ago

Why should I learn rust?

20 comments



r/rust · 1 mo. ago

Rust doesn't belong in the Linux kernel;

75 comments



r/rust · 4 mo. ago

Can Rust actually replace C, considering pretty much all other languages are C-based?

26 comments



r/rust · 7 yr. ago

Using libc crate in Rust?

16 upvotes · 16 comments



r/rust · 2 yr. ago

Compiling against specific version glibc

14 upvotes · 17 comments



r/Gentoo · 5 yr. ago

gcc vs clang vs musl, what is (usually) the fastest?

13 upvotes · 10 comments



r/rust · 8 mo. ago

Building binary for old glibc versions

3 upvotes · 19 comments



r/rust · 2 yr. ago

A toy std::fs impl that does not need libc

264 upvotes · 58 comments



2 upvotes · 2 comments



r/rust · 7 yr. ago

What is libc crate/package for?

20 upvotes · 15 comments



r/rust · 9 yr. ago

Could a libc implementation be written in Rust?

76 upvotes · 28 comments



r/C_Programming · 5 yr. ago

Which libc is better as a reference to study implementation detail?

3 upvotes · 6 comments



r/rust · 6 mo. ago

What OS are you using with Rust and how has the experience been like?

27 upvotes · 114 comments



r/rust · 2 mo. ago

Should I learn Rust

6 upvotes · 58 comments



r/C_Programming · 3 yr. ago

What is glib? What situations is it useful?

29 upvotes · 16 comments

TOP POSTS



Reddit

reReddit: Top posts of December 16, 2018



Reddit

reReddit: Top posts of December 2018



Reddit

reReddit: Top posts of 2018



Log In
